YOUR CLOSET, STYLED

What do I wear *today?*

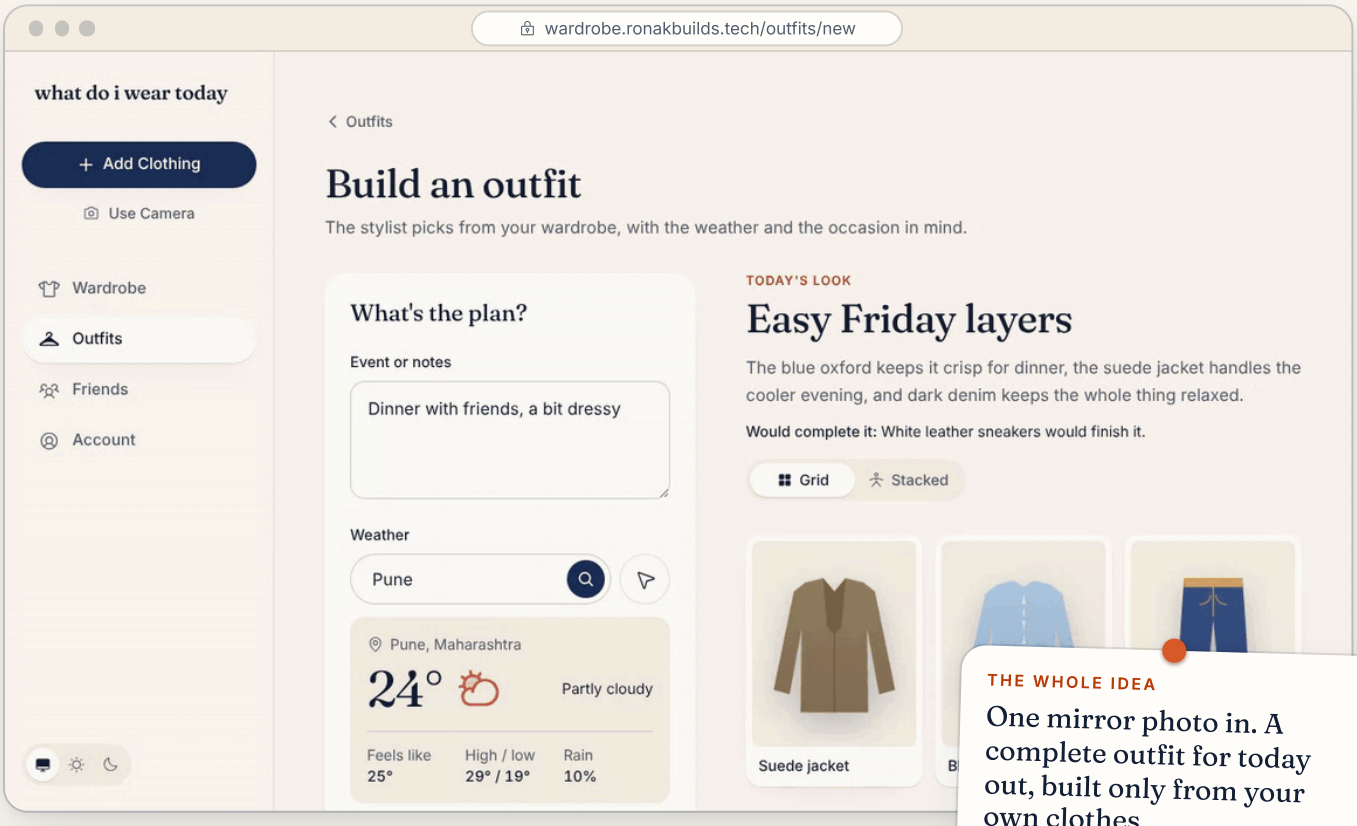
Snap the clothes you already own. An AI stylist builds outfits from them for the weather, your plans and your friends. Nothing new to buy.

BUILT BY
Ronak Pal
ronakpalforyou@gmail.com

TRY IT
wardrobe.ronakbuilds.tech

CODE
github.com/fourtysevencode/what-do-i-wear-today

STATUS
Public Beta
Shipped to Hack Club Terra



THE WHOLE IDEA

One mirror photo in. A complete outfit for today out, built only from your own clothes.

INSIDE

01 The problem
Challenge 1
Why it matters 2

02 The first AI prompt
Challenge 2
Mentor in a box 4

03 Product blueprint
Assignment 3
Flow, parts, scale 5

04 The app today
Assignment 4
New, built, next 7

05 Go-to-market
Assignment 5
Launch and grow 10

A full closet, and nothing to wear.

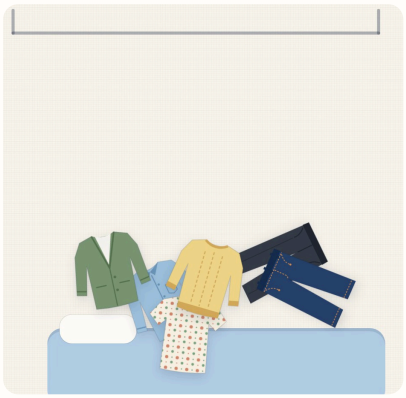
Getting dressed is a decision almost everyone makes every single day, and it's still oddly hard.

01



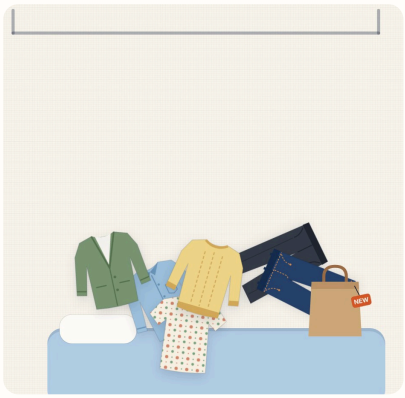
7:48 am

A closet full of clothes, and nothing feels right.



8:09 am

Half of it you forgot you owned. The rest ends up on the bed.



And then

You buy something new. The outfit was in your closet all along.

Stills from the product's 30-second explainer film (docs/explainer.mp4).

1 What problem have you personally noticed?

Most mornings I stand in front of a wardrobe full of clothes and still feel like I have nothing to wear. I forget what I own, so I keep rotating the same few pieces, and I only think about the weather after I've already picked something. It gets harder when I'm going somewhere with friends: we send mirror selfies back and forth in the group chat, trying to work out who's wearing what so we look like we planned it. Sometimes buying something new feels like the easiest fix, even though a good outfit was already hanging in my closet.

2 Who else has this problem? How do you know?

Anyone who gets dressed every day and owns more than a handful of clothes, and especially **students and young professionals**, who dress for classes, work, dinners, fests and trips in the same week. I know because it's my own problem every morning, and I see the same thing in the people around me: before any plan, the group chat fills up with "what are you wearing?" and photos of outfits laid out on a bed. "I have nothing to wear" is one of the most common things people say, usually in front of a full wardrobe.

Every day

It's a daily decision, so even a small fix is felt every morning.

Many occasions

College, office, dinner, a fest and a trip can all land in one week.

Never alone

Plans with friends turn one outfit into a group decision.

"I have nothing to wear," said in front of a wardrobe full of clothes.

The gap between owning clothes and using them is the whole opportunity.

3 What do people do today, and why isn't it good enough?

WHAT PEOPLE DO	WHY IT FALLS SHORT
Try things on at the mirror	Slow, and you only consider what you can see or remember. The rest of the closet is invisible.
Scroll Pinterest or Instagram	The inspiration uses clothes you don't own, which quietly pushes you to buy more.
Check a weather app	A separate step that usually happens after the outfit is already chosen.
Coordinate in the group chat	Photo by photo, and nobody can see the other person's whole wardrobe.
Use a digital closet app	Mostly built for one person, and adding a wardrobe usually means photographing and tagging pieces one by one.

Each tool solves one slice. None of them look at your clothes, today's weather, your plans and your friends at the same time.

4 What is one possible idea to solve it?

What do I wear today?: a digital wardrobe with an AI stylist that only works with clothes you already own.

01 · SNAP

Take one mirror photo

A segmentation model finds every piece, cuts it out and detects its colours.

02 · STYLE

Say what today is for

"Dinner, a bit dressy" plus the live weather becomes a complete outfit from your rack.

03 · MATCH

Plan it with a friend

Two wardrobes, one coordinated pair of looks. Each person wears their own clothes.

5 Why do you think this idea could be viable?

A daily habit is built in
Everyone gets dressed every morning, so the app has a natural reason to be opened every day.

Cheap to run
A CPU model, Gemini Flash-Lite and serverless hosting keep the cost of an outfit to a fraction of a cent.

Growth through friends
Matching needs a friend, so every engaged user has a real reason to invite someone.

The timing is right
"Buy less, wear what you own" fits a growing push to re-wear clothes, and the app is already live in public beta.

★ Bonus: how is it different from existing solutions?

One photo, many pieces
A single mirror selfie adds the top and the bottom. No flat-lays, no manual tagging.

Only your clothes
The stylist can only pick from the pieces you own. It never invents an item or sells you one.

Weather plus plans
The live forecast and your note go into the same request, so the outfit fits the actual day.

Built for two
Matching looks from two real wardrobes, for the plans you make together.

A mentor in a box.

Before writing code, I used one long prompt to get an experienced founder's view of the whole journey, from testing the problem to the first users. This is that prompt, rewritten for this product.

02

ROLE
Who to be

Act as a serial entrepreneur with 10 to 20 years of experience who has built and grown consumer apps, and as a product designer who understands fashion and everyday style. You also understand computer vision and large language models well enough to tell me what is realistic to build.

GROUND RULES
No fluff

Your job is to be my end-to-end startup mentor and product partner. Don't give me generic startup advice. Think practically about what a real person would actually use every morning, and what would make them come back.

THE PRODUCT
Make it concrete

I want to build “What do I wear today?”, a digital wardrobe with an AI stylist. Users take one mirror photo; a segmentation model cuts out each garment, names it and detects its colours. The user then describes their plans, such as “dinner with friends, a bit dressy”, and the app suggests a complete outfit using only clothes they already own, taking the live weather into account. Friends can connect wardrobes and get coordinated outfits for a shared event, each person wearing their own clothes.

STEP 1
Validate

First, identify the exact problem and the best first target user, and list the assumptions I need to validate. Tell me how to test them by talking to people and trying the workflow before I build too much.

STEP 2
Design the MVP

Then design a focused MVP: what to build first, what NOT to build yet, and the ideal flow from sign-up to the first outfit. Explain how the pieces fit together: the camera, the segmentation model, colour detection, weather, the AI stylist, storage for clothing photos and the friends feature.

STEP 3
Early traction

Next, help me decide on pricing (or whether to stay free at first), how to get the first 100 users, how to collect feedback, and how to measure whether people actually get dressed with the app. List competing or substitute solutions I should study, such as closet apps, Pinterest, shopping apps and the group chat, and what to learn from them without copying them.

STEP 4
Roadmap & risks

Finally, give me a step-by-step roadmap covering validation, design, development, testing, launch and early growth, with the outcome I should reach before moving on at each step. Point out the biggest risks: model accuracy on real mirror selfies, AI cost and rate limits, privacy of personal photos, and retention once the novelty wears off. Tell me the reasons this could fail, with ways to test or reduce each one.

Give it a role

Asking for a founder *and* a stylist *and* an AI realist gets advice from all three angles at once.

Describe the real product

A concrete flow (photo, cut-outs, plans, weather, friends) stops the answers from being generic.

Ask for an outcome per step

Each stage has to end in something I can check before moving on, not just a to-do list.

Name the scary parts

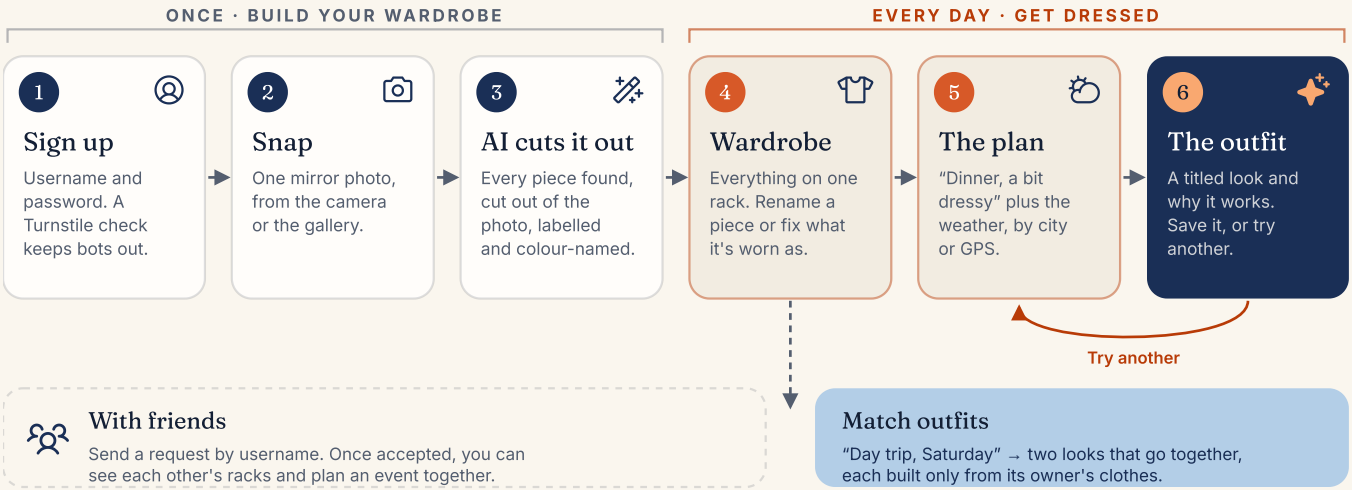
Listing accuracy, cost, privacy and retention forces honest answers about how this could fail.

From mirror selfie to tomorrow's outfit.

The main user flow, the four building blocks behind it, and what happens if 10,000 people show up tomorrow.

03

The main user flow



The four building blocks, in my own words

Frontend

The shop floor

A **Next.js** app on **Vercel**. It's every screen you touch: the wardrobe, the outfit builder, friends. Its server side checks who you are before anything private happens, and it's the only part that talks to the database and the AI backend, so the browser never sees those addresses or keys.

Backend

The workshop

A **Python FastAPI** service in a **Docker** container on **Hugging Face Spaces**. It does the heavy lifting: running the segmentation model, finding colours, fetching the weather and asking Gemini for outfits. The costly endpoints need a shared secret token.

Database & storage

The memory

Postgres on Neon remembers users, garments, friendships and saved outfits. Cut-outs live in a **private Backblaze B2** bucket: you only get a 5-minute link to a picture if it's yours or a friend's. The original photo is never stored.

AI

The eye and the stylist

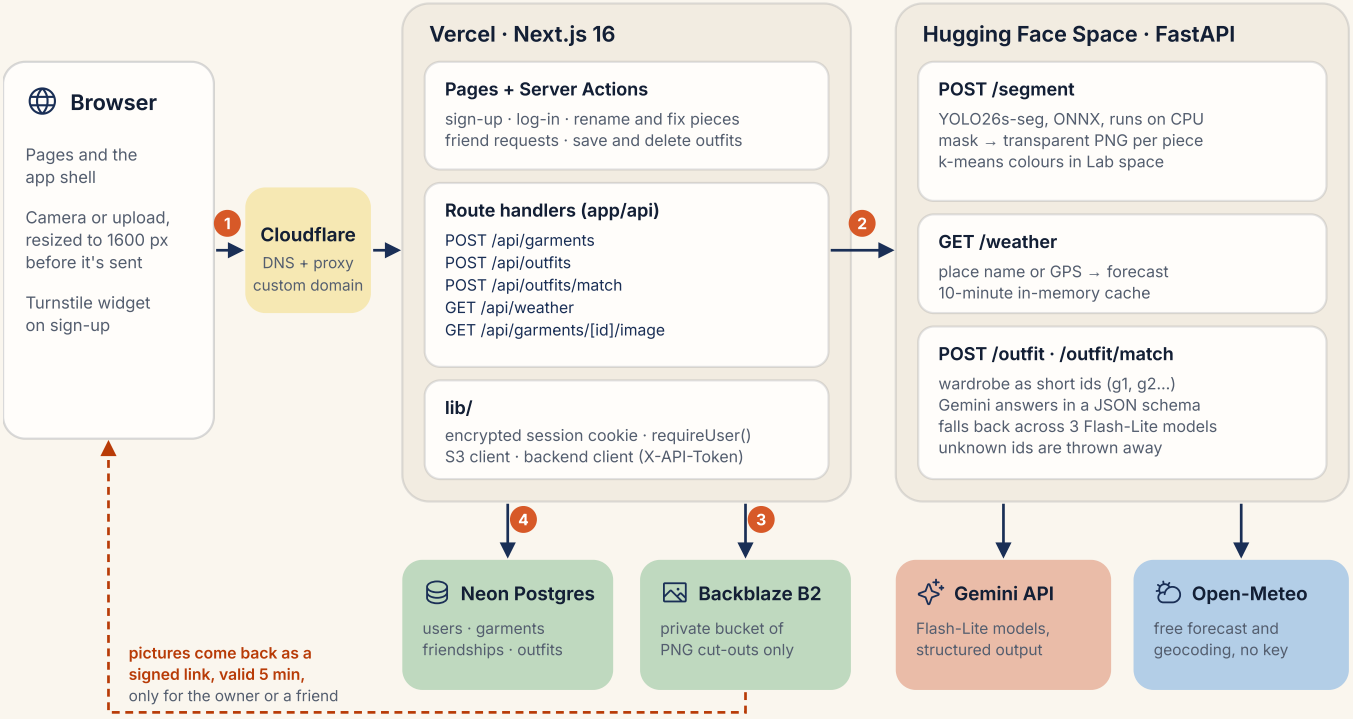
Two different AIs. The **eye** is a YOLO segmentation model trained on 13 clothing classes: it finds each garment and its exact outline, and k-means clustering names its main colours. The **stylist** is **Gemini Flash-Lite**: it reads your wardrobe as a short list, plus the weather and your note, and answers with an outfit in a fixed JSON shape.



A real prediction from the trained model on a mirror selfie: each piece gets a mask, a label and a confidence. The masked pixels become a transparent cut-out.

How the blocks connect

Adding a piece follows the numbered path. Everything private goes through the Next.js server, and pictures come back only as short-lived signed links.



Adding a piece: ① the resized photo goes to `/api/garments` · ② which sends it with the token to `/segment` · ③ each cut-out is saved to the private bucket · ④ its label and colours become a row in Postgres. The photo itself is thrown away.

What breaks if 10,000 people use this tomorrow?

in the order it would break

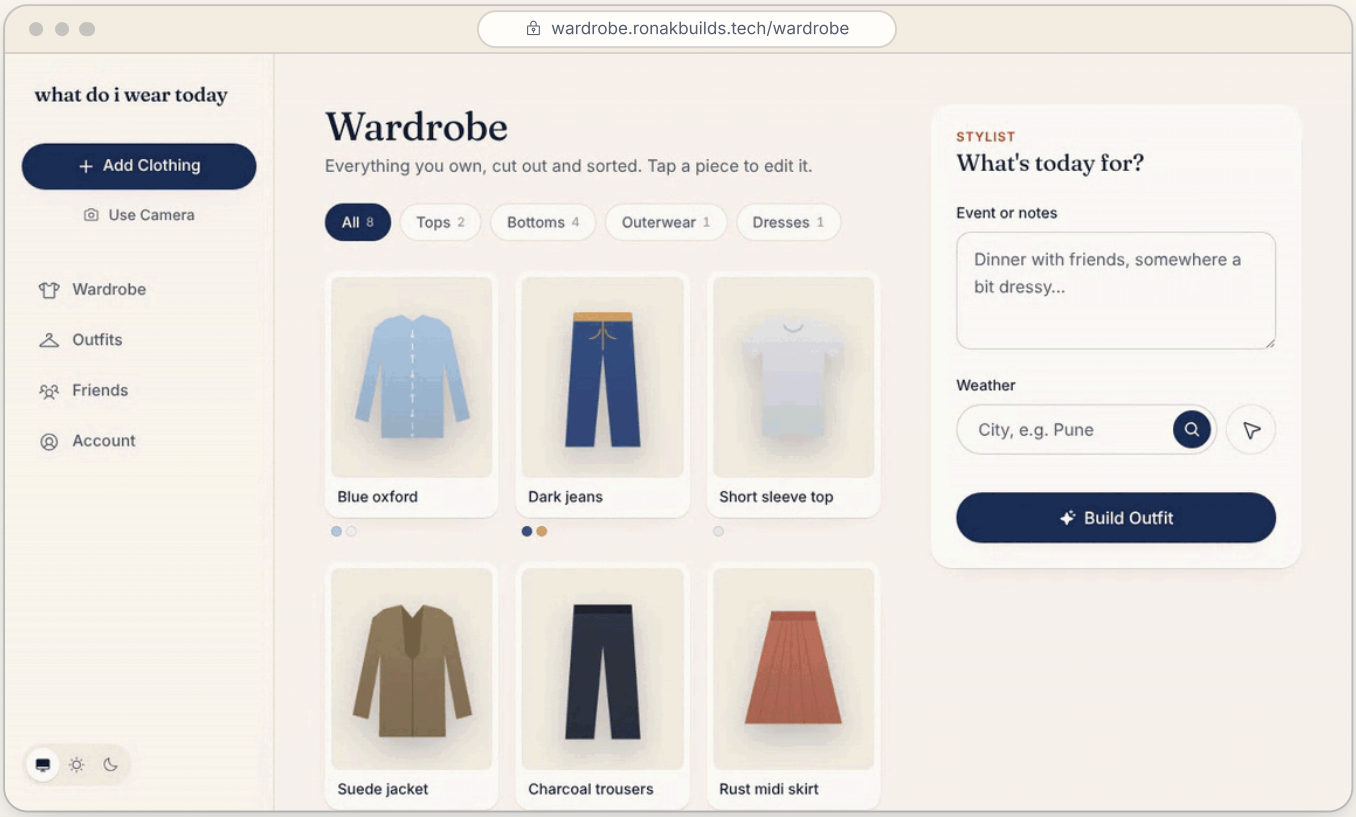
Short answer: the backend first, then the AI quota. Vercel scales on its own; one free CPU container running a vision model does not.

PART	WHAT HAPPENS AT 10,000 USERS	HOW I'D FIX IT
BREAKS Segmentation	Every new user's first photo queues behind one free CPU container. Requests pass the 55-second limit and fail with "the server is waking up". Free Spaces also sleep when idle, so the first visitor waits for a cold start.	Paid always-on hardware with several replicas (or a GPU endpoint), plus a job queue: accept the photo instantly, show "cutting out…", and fill the wardrobe when it's done.
BREAKS Gemini quota	Each "Build outfit" is one Gemini call. The 3-model fallback buys time, but once every model returns 429 people see "the stylist is busy". There's no per-user limit yet, so one script could burn the whole quota.	A paid tier, a per-user daily limit on outfit requests, and caching repeat requests for the same wardrobe, plan and weather.
STRAINS Weather	Open-Meteo's free tier is meant for non-commercial use under 10,000 calls a day. The cache lives in one container's memory and is lost on every restart.	A shared cache (e.g. Redis) keyed by place and hour, and a commercial plan once usage is real.
STRAINS Images	Each cut-out is fetched through a fresh, uncached signed link, so every wardrobe visit re-downloads every PNG. At 20 pieces each, that's hundreds of thousands of downloads a day.	A CDN in front of the bucket, longer-lived links the browser can cache, and small thumbnails for the grid.
HOLDS The rest	Vercel functions scale automatically, sessions are stateless encrypted cookies, Neon's HTTP driver copes with spikes (on a paid plan), and Turnstile keeps bot sign-ups out.	

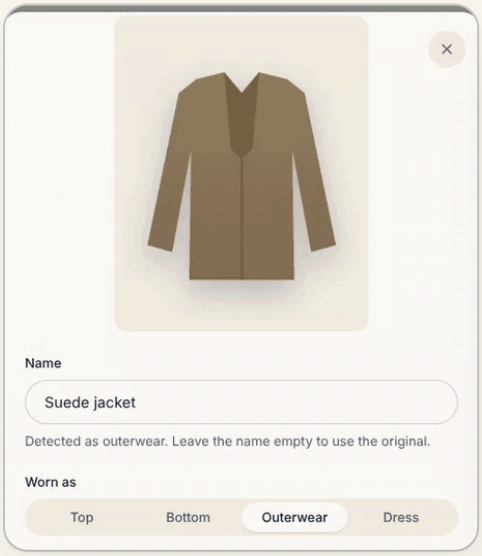
Live, in public beta.

What's new in this version, how it's built, and what comes next. Try it at **wardrobe.ronakbuilds.tech**.

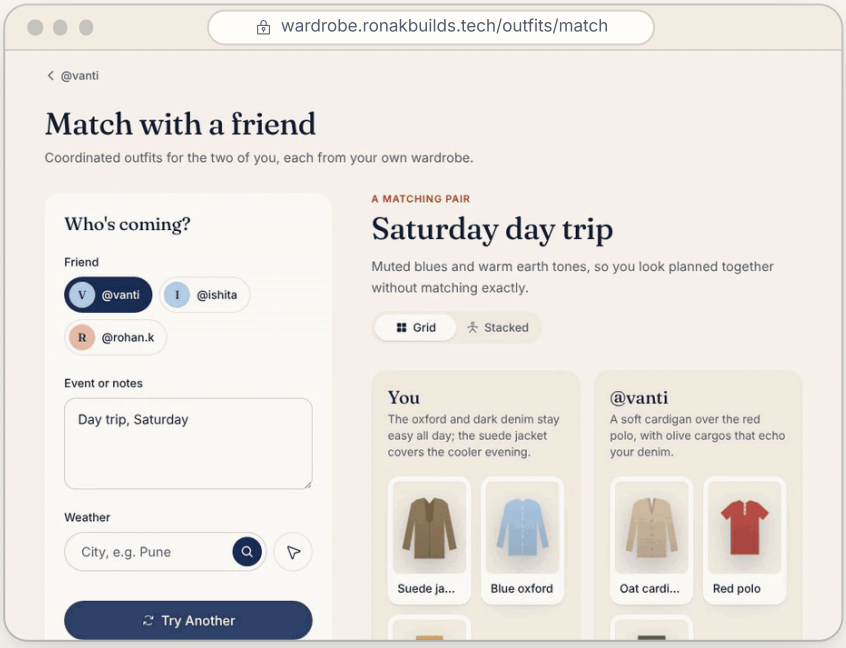
04



New: the Linen & Denim redesign. The wardrobe with filters for tops, bottoms, outerwear and dresses, and the stylist card always to hand. On a phone, the sidebar turns into an iOS-style tab bar with a camera button in the middle. Shown with a sample wardrobe.



New: fix the AI when it's wrong. Rename a piece or change what it's worn as. The stylist uses your correction from then on.



New: match with a friend. "Day trip, Saturday" becomes a pair of looks in a shared palette, each one from its owner's wardrobe.

What's new in this version

- **Linen & Denim redesign** with an iOS-style app shell: tab bar and sheets on phones, a sidebar on desktop, and light, dark or system themes.
- **Piece editor:** rename a piece, set what it's worn as (top, bottom, outerwear or dress) and remove colours the model got wrong.
- **Camera capture** straight from the app. Photos are resized in the browser to keep uploads small and fast.
- **AI stylist** with live weather by city name or GPS, and a grid or stacked view of the outfit.
- **Saved outfits** and **matching outfits** with accepted friends.
- **Accounts** with username and password, Cloudflare Turnstile, and a Privacy Policy and Terms you agree to at sign-up.
- **A status page** that checks the backend and says when it's waking up.

BY THE NUMBERS

< 1 week
from the first backend commit (4 Oct) to the public beta (9 Oct)

13
clothing classes the model recognises

~74%
mAP@50 after 82 training epochs

~2 s
typical Gemini answer for an outfit

5
backend endpoints. The 3 costly ones need a secret token.

0
original photos stored. Only the cut-outs are kept.

Build log

4 Oct
MVP written down. FastAPI backend started.

6–7 Oct
Vision pipeline tested in notebooks. Next.js frontend started.

8 Oct
ONNX segmentation endpoint, homepage, accounts, camera, friends, weather, Gemini outfits and matching.

9 Oct
Privacy policy and terms, real photos on the landing page. **Public Beta.**

14 Oct
Linen & Denim redesign, iOS-style shell, new demo and explainer film.

How it's built

FRONT END

Next.js 16 on Vercel

- React 19 Server Components and Server Actions; Tailwind CSS 4
- Login with bcrypt-hashed passwords and an encrypted `iron-session` cookie
- Route handlers proxy every backend call, so its URL and token stay server-side
- Cloudflare in front of the custom domain

BACK END

FastAPI in Docker on HF Spaces

- YOLO26s-seg exported to ONNX, run on CPU with ONNX Runtime
- OpenCV k-means in Lab space for up to 3 colours per piece
- Gemini Flash-Lite with a JSON schema and a 3-model fallback
- Open-Meteo for weather, cached for 10 minutes

DATABASE & STORAGE

Neon Postgres + Backblaze B2

- Six tables: users, garments, friendships, outfits, outfit items and a usage log
- Neon's serverless driver over HTTP, built for serverless functions
- Cut-outs in a private B2 bucket via the S3 API, 5-minute signed links only

The backend API

`https://fourtysevencode-what-do-i-wear-today.hf.space`

METHOD	PATH	AUTH	WHAT IT DOES
GET	`/health`	none	Status, uptime and version (“Public Beta”)
POST	`/segment`	token	Cuts every garment out of a photo and detects its colours
GET	`/weather`	none	Current weather and today's range, by `?q=Pune` or `?lat=&lon=`
POST	`/outfit`	token	Picks one outfit from a wardrobe (Gemini)
POST	`/outfit/match`	token	Coordinated outfits for two people, each from their own wardrobe

Under the hood: three pieces of real code

One photo in, every piece out

`POST /segment`

```
curl -X POST "$BASE_URL/segment" \
  -H "X-API-Token: $API_TOKEN" \
  -F "image=@mirror-selfie.jpg"

# 200 OK (trimmed example)
{
  "count": 2,
  "items": [{
    "label": "long sleeve top",
    "confidence": 0.912,
    "colors": [
      { "name": "navy",  "hex": "#1f2a44", "percentage": 71.4 },
      { "name": "white", "hex": "#f2f2f0", "percentage": 22.0 }
    ],
    "image": "<base64 PNG, cut out to the mask>"
  }, …]
}
```

The stylist can't invent clothes

`backend/services/stylist.py`

```
# What Gemini actually reads
Occasion / notes: Dinner with friends, a bit dressy
Weather: Pune, Maharashtra, India: 24°C (feels 25°C), …
Their wardrobe:
- g1: Blue oxford (long sleeve top) (light blue)
- g2: Dark jeans (trousers) (denim, navy)

def _resolve(ids, back):
    """Keep only ids that were offered,
    in order, without repeats."""
    seen, out = set(), []
    for short in ids or []:
        real = back.get(str(short).strip())
        if real and real not in seen:
            seen.add(real)
            out.append(real)
    return out
```

Short ids (g1, g2… and f1, f2… for a friend) keep the prompt small and keep each person to their own clothes. Gemini must answer in a fixed JSON shape (title, item ids, reasoning, missing piece), and any id it wasn't given is dropped.

Private by default

`app/api/garments/[id]/image/route.ts`

```
// condensed from the real handler
const user = await getCurrentUser();
if (!user) return new NextResponse(null, { status: 401 });

// Same 404 for "doesn't exist" and "not yours",
// so ids can't be probed.
const key = await storageKeyForViewer(id, user.id);
if (!key) return new NextResponse(null, { status: 404 });

// Owner or accepted friend: a 5-minute signed link
return NextResponse.redirect(await signedGetUrl(key, 300), 302);
```

TRY THE BETA

Scan, sign up and add your first photo.

wardrobe.ronakbuilds.tech
Works in any phone browser, camera included. The free backend can take a minute to wake up on the first photo.

What I'll add next

01

Shoes and accessories

The model's 13 classes have no shoes, which is why the stylist sometimes says “white sneakers would finish it”. Next: train on shoes, bags and accessories.

02

Uploads that never time out

An always-on backend and a job queue, so a photo is accepted instantly and the cut-outs appear when they're ready.

03

Outfit calendar

Log what you wore, so the stylist avoids repeats and you can see the pieces you never reach for.

04

Group matching

More than two people for trips, weddings and fests, with one shared theme.

05

A morning outfit, delivered

Install it as an app and get today's look and the forecast before you open the wardrobe.

06

Share cards and limits

Export a look as an image for stories, and add per-user limits that protect the AI quota.

From built, to worn every morning.

The plan for turning a working app into one people discover, try and use: who it's for, why they'd pick it, and how it reaches them.

05

1 Target audience: who is it built for?

The first audience is college students aged about 17 to 24, starting with my own college and my friends' colleges. They dress for many different occasions in one week, they live in friend groups that already coordinate outfits, they're phone-first and comfortable with AI, and they're on a budget, so “use what you own” lands. They're also easy to reach through campus groups and fests.

A

Aanya, 20

EXAMPLE PERSONA

Second-year student, lives with a big friend group

HAS	A full wardrobe and a group chat that votes on every outfit.
TODAY	Twenty minutes at the mirror and three outfits on the bed.
WANTS	To look put together for Friday's fest without buying anything.

NEXT, ONCE IT WORKS ON CAMPUS

- **Young professionals** after “office, but not too formal”
- **Buy-less shoppers** who want to re-wear more
- **Travellers** packing one bag for a trip
- **Friends and couples** dressing for weddings and festivals

2 Value proposition: what it solves, and why pick it

For people who own plenty but feel they have nothing to wear, *What do I wear today?* turns one mirror photo into a digital wardrobe and builds outfits for the weather, your plans and your friends, using only the clothes you already own.

WHY SOMEONE WOULD CHOOSE IT

Fast to start

One photo adds several pieces. Most closet apps make you add them one by one.

Fits the actual day

Weather and plans in one ask, not three apps and a mirror.

Social by design

Matching looks from two real wardrobes, which Pinterest and the group chat can't do.

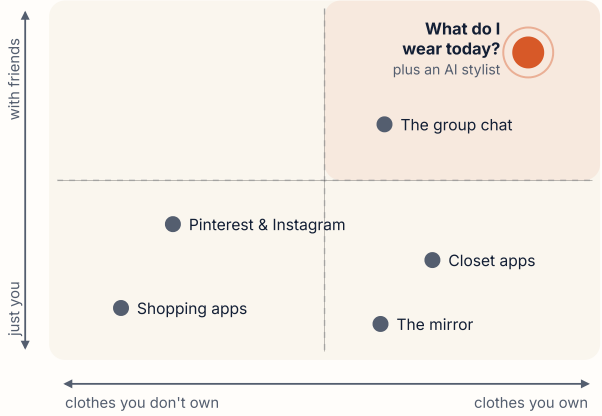
Honest and private

It never sells you clothes, and it never keeps your original photo.

IN ONE LINE

Wear what you already own, and look like you planned it together.

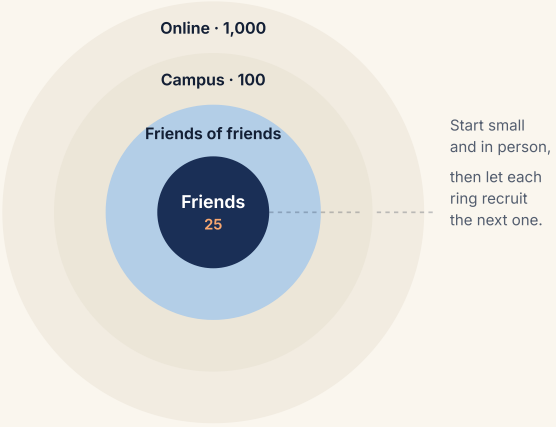
WHERE IT SITS



How I see the landscape. The group chat is the closest substitute, but it has no view of the whole wardrobe and no stylist.

3 Acquisition strategy: the first users

Start where the problem is loudest and I can watch people use it: in person, with friends, then outwards one ring at a time. Matching is the engine, because every match needs a second person with a wardrobe.



RING	HOW THEY ARRIVE	TARGET
Friends	I set up each wardrobe with them in five minutes and watch where they get stuck.	25 wardrobes, week 2
Their friends	Every match sends an invite: “match outfits with me for Saturday”.	1 invite per user
Campus	A fit-check booth at a fest and ambassadors in fashion and cultural clubs.	100 by week 6
Online	Short videos, communities and a public launch (below).	1,000 by week 12

THE PATH EVERY NEW USER TAKES

Discover — sees a Reel, a story card or a match invite

Sign up — username, password, done in seconds

First photo — one mirror selfie fills the rack

First outfit — the “aha”: a look for today, from their own clothes

Invite — asks a friend to match for a real plan

4 Marketing channels, and why each one

CHANNEL	WHY IT FITS	WHAT I'LL DO	WHAT I'LL MEASURE
Instagram Reels & YouTube Shorts	Fashion is visual, and before and after videos travel.	A series: “I let an AI dress me from my own closet for 7 days”, plus matching looks with a friend.	Views → sign-ups through a tracked link
WhatsApp & campus groups	It's where outfit planning already happens.	Share outfit cards and the match link before fests, trips and parties.	Invites sent, and how many become sign-ups
College fests & clubs	Moments when everyone is planning what to wear.	A fit-check booth: snap a photo, get a matching look with a friend on the spot.	Sign-ups per event
Reddit, Discord & Hack Club	Early adopters and builders give honest feedback.	Build-in-public posts with the demo GIF, following each community's rules.	Feedback threads, beta users
Product Hunt & LinkedIn	A credible launch spike and reach beyond campus.	A launch post built around the 30-second explainer film.	Upvotes, traffic, creator interest
Thrift and buy-less creators	Their audience already believes “wear what you own”.	Early access and a co-hosted closet challenge.	Sign-ups per creator link

BUDGET

₹0 to start

Friends, campus and organic video first. Paid ads only once people stick around.

CADENCE

3 short videos a week

Real outfits from real wardrobes, filmed in a few minutes on a phone.

FOCUS

Double down on what works

Every link is tracked, so after a month the best two channels get all the time.

5 Launch plan: from beta to traction

Four phases, each with a clear test to pass before moving on.

WEEK 0

Get ready

- Always-on backend and per-user limits
- Events for sign-up, first photo, first outfit, first match
- A sample wardrobe to try before uploading

MOVE ON WHEN
a new user gets a first outfit in under 2 minutes

WEEKS 1–2

Private beta

- 25 friends, onboarded in person
- A short weekly chat: what did you wear, and why?
- Fix the top three points of friction

MOVE ON WHEN
4 in 10 still build outfits in their second week

WEEKS 3–6

Campus launch

- A fit-check booth at a college fest
- Ambassadors in 3 colleges
- Matching invites switched on

MOVE ON WHEN
100 active wardrobes, and each user invites at least one friend

WEEKS 7–8

Public launch

- Product Hunt and a LinkedIn post
- The “7 days dressed by AI” Reels series
- The explainer film everywhere

GOAL
1,000 sign-ups and a steady weekly habit

NORTH STAR
Outfits built per active user, per week

ACTIVATION
5+ pieces and one outfit in the first session

RETENTION
Still building outfits after 7 and 30 days

SPREAD
Match invites sent per user

The app already logs every outfit the stylist builds, solo or matched, so the north star can be tracked from day one.

6 Growth strategy: what keeps it growing

Three loops that feed each other, so growth doesn't depend on me posting every day.

Friend loop

1. You plan an event and tap Match
2. Your friend gets an invite and signs up
3. They add their wardrobe to match you
4. They match with their own friends

↻ back to 1

Morning loop

1. Today's look and forecast arrive at 7:30
2. You wear it, or ask for another
3. The calendar learns what you wear
4. Tomorrow's suggestion gets better

↻ back to 1

Share loop

1. A matching pair becomes a story card
2. Friends see it, with a link
3. They try it with one photo
4. Their first look becomes a card too

↻ back to 1

Widen the use cases

Trip packing lists, wedding and festival groups, and office-wear for young professionals, then more cities and languages.

Make money without breaking the promise

The core stays free. A Pro tier can add the calendar, packing lists and group matching. No paid placements: if shopping ever appears, it's opt-in, only for the missing piece the stylist names, and second-hand first.

THE PLAN IN ONE SENTENCE

Win one campus, let matching spread it friend to friend, and earn the morning habit.

wardrobe.ronakbuilds.tech · github.com/fourtysevencode/what-do-i-wear-today